



CarePortal



Ferduse Rahman, Soogeun Kim, Mohammad
Bhuiyan, Genesis Castillo



The Problem

- Patients face challenges accessing their and managing their healthcare data in a easy and manageable way.
- Booking appointments, tracking medical history and understanding potential health risks has to be done in person with a doctor or never at all.
- Most hospitals lack a easy digital way for patients to manage their data

Why does this matter

- Healthcare is one of the most critical sectors needing digitization
- Early disease-risk prediction can help patients become more aware of possible health concerns, encouraging them to be more proactive about scheduling doctor visits and seek medical guidance before problems become more serious.
- Automates more work for hospitals
- Improves patient experience and engagement through an easy to use interface

The Solution

- Design and build a user friendly web portal where patients can register, login, book appointments, and enter and view their medical data.
- Create a database to store and secure all patient medical data.
- Create an ML model that can predict if a person is at risk for a *specific* disease on medical data, age, and other factors.

Major Features

- Secure patient registration and login
- Dashboard to view and manage patient data
- Appointment booking and tracking
- Entry for medical history
- ML powered disease risk prediction
- Prediction results display

Technologies Used/ System Architecture

1. Patient Web Interface: HTML/CSS/JavaScript



2. Backend API Layer: Python + FastAPI

- Also connects to ML Prediction Model which returns risk prediction for a “specific” disease



3. ML Prediction Model: Pytorch / scikit-learn



4. Database Layer: MySQL/PostgreSQL (Patient Records, Appointments, Medical History, Predictions)

System Flow

1. Patient enters information on the frontend
2. Frontend sends request to backend API
3. Backend validates the data
4. Backend stores or retrieves data from the database
5. Backend sends health inputs to the ML model
6. Backend returns scheduled appointment, medical history, and prediction results to the dashboard

Genesis Castillo: Frontend Developer

Primary Responsibilities:

- Design and implement the patient-facing web interface.
- Create registration and login pages.
- Build the patient dashboard for appointments, medical history, and prediction results.
- Create appointment booking and appointment history pages.
- Build the medical history form and prediction result page.
- Work with the Backend Integration Developer to connect forms and pages to API endpoints.

Technologies Used:

- HTML
- CSS
- JavaScript
- Git/GitHub

Expected Deliverables:

- Patient web interface
- Appointment booking page
- Medical history form
- Prediction result display page

Genesis Castillo: Frontend Developer

Pages to Build:

- Login/Register
- Patient Dashboard
- Appointment Booking and Tracking
- Medical History
- Prediction Results

Ferduse Rahman: ML Engineer

Primary Responsibilities:

- Select a small healthcare dataset.
- Use patient input features such as age, BMI, glucose, blood pressure, symptoms, or other health-related values.
- Train a disease-risk prediction model using Pytorch.
- Output a risk level such as low, medium, or high
- Evaluate the model using accuracy, recall, specificity, and confusion matrix.

Technologies Used:

- Python
- Pandas
- scikit-learn
- PyTorch
- Git/GitHub

Expected Deliverables:

- ML training code
- Trained prediction model
- Evaluation results
- Risk prediction API connection

Ferduse Rahman: ML Engineer

Steps to Success:

1. Find and process dataset
2. Build the model
3. Train the model
4. Test and evaluate the model
5. Integrate model with API

Mohammad Bhuiyan: Database Engineer

Primary Responsibilities:

- Design the database structure.
- Create an ER diagram showing relationships between major entities.
- Design tables for users, patients, doctors, appointments, medical history, symptoms, and predictions.
- Define primary keys, foreign keys, and table relationships.
- Write SQL scripts and sample data.
- Write queries for appointments, patient history, doctor availability, and prediction history.

Technologies Used:

- SQL
- MySQL or PostgreSQL
- ER diagram tool such as draw.io, Lucidchart, or diagrams.net
- Git/GitHub

Expected Deliverables:

- ER diagram
- Relational database schema
- SQL creation scripts
- Sample data
- SQL query examples

Mohammad Bhuiyan: Database Engineer

Steps to Success:

1. Design ER Diagram
2. Define the full Schema
3. Write SQL scripts to create tables
4. Write Sample data for testing
5. Create queries for endpoints

Soogeun Kim: Backend Integration Developer

Role Overview:

- My role is to build the backend API layer for the healthcare web portal.
- The backend connects the patient interface, database, and machine learning model.
- I will make sure each part of the system can communicate correctly.

Main Responsibilities:

- Build the Python backend server using FastAPI
- Create REST API endpoints for patient registration, login, appointments, medical history, and disease-risk prediction.
- Connect frontend pages to backend APIs.
- Connect backend logic to the database.
- Integrate the ML model into the /predict-risk endpoint.
- Add input validation, error handling, and integration testing.

Backend APIs and System Integration - Soogeun Kim

Planned API Endpoints:

- /register: Create patient accounts
- /login: Allow patients to access their account
- /appointments: Retrieve appointment information
- /book-appointment: Allow patients to schedule appointments
- /medical-history: Store and retrieve patient medical information
- /predict-risk: Send patient health data to the ML model and return a risk result

How My Part Connects the System:

- Frontend sends patient data to the backend.
- Backend validates the data.
- Backend saves or retrieves information from the database.
- Backend sends health inputs to the ML model.
- Backend returns results to the patient dashboard.

Technologies, Deliverables, and Challenges - Soogeun Kim

Technologies:

- Python
- FastAPI
- REST APIs
- MySQL or PostgreSQL

Expected Deliverables:

- Working backend server
- Documented REST API endpoints
- Database connection layer
- ML model integration

Expected Challenges:

- Making sure the frontend, backend, database, and ML model communicate correctly
- Handling invalid or missing patient input data
- Connecting the backend to the database without data errors
- Integrating the ML model once it is completed

Healthcare Appointment and Diagnostic Assistant

MVP Prototype Flow



Timeline

June 18: Finalize proposal, slides, technology stack, team responsibilities, GitHub repository, mockups, and system architecture diagram.

June 19 - June 30: Design ER diagram and database schema; create initial frontend pages; set up backend server and simple API routes; begin dataset selection.

July 1 - July 7: Implement patient registration and login pages; build register and /login endpoints; connect basic frontend forms to backend APIs; begin dataset preprocessing.

July 8 - July 14: Implement appointment booking and appointment history; create /appointments and /book-appointment endpoints; connect appointment APIs to the database; continue ML training.

July 15 - July 21: Implement medical history form and backend endpoint; store/retrieve patient medical history; finalize first version of ML prediction model; define model input/output format.

July 22 - July 28: Integrate the ML model into /predict-risk; display prediction results on the patient dashboard; store prediction results in the database; begin integration testing.

July 29 - August 4: Fix integration bugs; improve validation and error handling; test login, appointment, medical history, and prediction workflows; improve UI consistency.

August 5 - Mid-August: Complete final testing, documentation, final demo

Expected Challenges

Technical Challenges

- Ensuring the frontend, backend, database, and machine learning model communicate correctly.
- Matching frontend form fields, backend API request bodies, database columns, and ML model inputs.
- Integrating the machine learning model into the web portal after the model is completed.
- Handling missing or invalid patient input before saving data or sending it to the model.

Team Coordination

- Each member is responsible for a different system component, so the team must coordinate API endpoint names, database fields, input formats, and page design.
- Regular communication and Git/GitHub version control will be used to reduce integration conflicts.

Security Concerns

- The system will handle patient-related information, so the team must avoid exposing sensitive data.
- The backend should validate input, handle errors clearly, and use basic secure password handling for login functionality.

Scalability Concerns

- The database should be designed to support more users, appointments, medical history records, and prediction results over time.
- The backend APIs should be organized so new features can be added without rewriting the entire system.

Testing and Integration Risks

- Because the project does not have a separate testing role, each member will test their own component.
- The Backend Integration Developer will focus on API and integration testing to verify that the frontend, backend, database, and ML model work together.

Q/A